Day 2 | Cost, monitoring and operations

Core attendee edition | 09:45-17:30

Investigate cost, requests, failures and dependencies.

Practise KQL and standalone queue operations.

Test fictional documents in a local-only tracker.

Use assigned resources; no deployment is required.

Today's route

09:45 Recap | 10:00 Cost investigation | 11:15 Break

11:30 Application Insights | 12:45 Lunch

13:30 KQL | 14:45 Break

15:00 Service Bus | 15:35 Local tracker

16:20 Incident and cost exercise | 17:00 Evidence and close

Follow Day 2\00_DAY_2_ORDER.docx, not file numbering.

Know what is implemented

Day 1 exercises test separate Azure components.

The HTTP Function returns a fixed fictional response.

Day 2 tracker uses local memory and a background worker.

It has no Azure clients, durable storage or sign-in.

Prepared telemetry does not come from the tracker.

AI and advanced Azure integration are outside this core pack.

Choose and label your evidence route

Live: approved resource, network path and data permissions.

Prepared KQL: fictional datatables in an authorised editor.

Offline: read CSV files and predict results; no KQL execution.

Record the actual route, scope, UTC time and outcome.

Overview access is not permission to read service data.

Keep keys, tokens and real data out of screenshots and notes.

Cost investigation | 10:00

Open 06_Cost_Management_Portal_Lab.docx.

Compare equal seven-day periods at an approved scope.

Group by service; identify the largest change.

Explain an optimisation and the operational risk.

Fallback: Labs\Day2Operations\fictional-costs.csv.

Budget design is core; creating a real budget needs approval.

Cost checkpoint | compare before combining

Use the fictional CSV to calculate each period total.

Which service changed, by how much and by what percentage?

Which read-only check would test your explanation?

These invented GBP amounts are not Azure prices or a quote.

Do not add different periods and call them one period's bill.

A budget alert does not automatically stop spending.

Application Insights | 11:30

Open 02_Application_Insights_Portal_Lab.docx.

Confirm the approved component and investigation time window.

Requests: volume, success, result code and duration.

Dependencies: downstream call, target, success and duration.

Use correlated operations to form a hypothesis.

No live collection? Label the prepared/offline route honestly.

Workspace and resource query schemas differ

Workspace: AppRequests / AppDependencies.

Fields include TimeGenerated, DurationMs and OperationId.

Application Insights: requests / dependencies.

Fields include timestamp, duration and operation_Id.

Choose the matching supplied query folder.

A missing table is different from zero matching rows.

KQL practice | 13:30

Open 03_KQL_Beginner_Hands_On.docx.

Choose Queries\Prepared, Workspace or ApplicationInsights.

Run file 01, then 02, then 03; each is self-contained.

Prepared time window: 28 October 2026, 10:00-10:05 UTC.

Use the KQL editor, not an assistant chat box.

Replace the resource-ID placeholder only for the live route.

Prepared KQL checkpoints

01: Five fictional requests.

02: Two failed requests, with HTTP 500 and 401.

03: Two dependencies over 1000 ms, correlated to requests.

OperationId links observations, not proof of root cause.

Live counts can differ; sampling and time windows matter.

CSV review predicts results but is not a passed query run.

Service Bus | why use a queue?

A producer sends a small message describing work.

A separate consumer processes and settles that message.

A queue buffers work and helps absorb bursts.

Message accepted does not mean business process completed.

Redelivery is possible; consumers need idempotent handling.

This lab is standalone, not connected to the local tracker.

Service Bus Explorer | 15:00

Open 01_Service_Bus_Portal_Lab.docx.

Use an assigned isolated queue with no competing consumer.

Send one fictional message; record its MessageId.

Peek: inspect without removing.

Receive: confirm the chosen mode and settle your own message.

Never purge a shared queue or consume another attendee's work.

Local document tracker | 15:35

Open LBS_Document_Tracker_Setup_And_Testing.docx.

Project: Labs\LbsDocumentTracker | requires .NET 10 SDK.

Upload fictional UTF-8 .txt files, at most 32 KiB.

Reference on the form must match SampleData\cases.json.

Local worker validates format, not credit or identity.

Keep the website on loopback; do not publish it.

Run the tracker checks first

From Labs\LbsDocumentTracker, run:

dotnet run --project ".\Checks" -- "$PWD\SampleData"

Pass the sample directory explicitly; do not omit the argument.

The root attendee notes also give the command from the pack root.

Expect 20 PASS lines and exit code 0; stop on failure.

These checks do not prove Azure integration or live delivery.

Start and test your local website

Open Labs\LbsDocumentTracker in a PowerShell terminal.

$env:ASPNETCORE_ENVIRONMENT = 'Development'

$env:DOTNET_ENVIRONMENT = 'Development'

dotnet run --urls "http://127.0.0.1:5193"

Open http://127.0.0.1:5193 and submit the ten supplied cases.

Keep the terminal running while you test.

Tracker outcomes and boundaries

Samples 01, 05 and 06: Completed.

Other seven samples: Needs review.

Wrong extension, oversized or invalid UTF-8: rejected upload.

/healthz: {"status":"ok"}

/readyz: {"status":"ready","mode":"local-memory"}

Restart clears records. Completed means valid training format.

Incident and cost investigation | 16:20

Open 04_Incident_And_Cost_Team_Exercise.docx.

A fictional submission is slow or failed; the bill has risen.

Establish scope, time and data source before drawing conclusions.

Separate observations, hypotheses and safe next checks.

Recommend an optimisation and explain its risk.

Solo attendees can write the same investigation individually.

Knowledge check | explain the evidence

Why does resource visibility not prove data access?

How is prepared KQL different from live telemetry?

Why does a queue send not prove business completion?

Why can two correlated events have another root cause?

Why is Completed not financial approval?

Why does a budget alert not stop resource charges?

Evidence, cleanup and completion | 17:00

Complete 03_MY_LAB_EVIDENCE.csv with your own results.

Use Passed, Blocked or Not run; identify live/prepared/offline.

Save non-secret commands, scope, time and expected/actual output.

Press Ctrl+C to stop your local servers after testing.

Do not delete shared resources or widen network/role access.

Keep the whole core pack together for later independent practice.